

The `reset()` method ❷ is called when starting a new round of the game. It clears the list of Baddies and resets the frame counter.

The `update()` method ❸ is where the real management of Baddies happens. Our intent here is to loop through all the Baddies, telling each one to update its own position and removing any that have fallen off the bottom of the window. However, there is a potential bug. If you simply iterate through a list and remove an element that matches your criteria for deletion, the list is immediately compacted. When this happens, the element directly following the one that was deleted will be skipped; in this loop, that element will not be told to update itself. Though I didn't go into detail at the time, we encountered the same problem in Chapter 11 in the Balloon game, where we needed to eliminate balloons that floated off the top of the window. There I employed a solution using the `reversed()` function applied to the list, to iterate in the reverse order (see [Listing 11-6](#)).

Here I've implemented a more generalized solution ❹. The approach used in the `BaddieMgr` class is to make a copy of the list and iterate over the *copied* list; then, if we find an element that meets the criteria for deletion (in this case a Baddie that has fallen off the bottom of the window), we remove that element (that specific Baddie) from the *original* list. With this approach, we are iterating over a different list than the one from which we are removing elements.

As we iterate through the Baddies, the call to the `update()` method of each Baddie ❺ returns a Boolean: `False` to indicate it's still moving down the window or `True` to indicate that it has fallen off the bottom. We count the number of Baddies that fall off the bottom and remove each one from the list. At the end of the method, we return the count to the main code so it can update the score.

In every frame, we also check to see if it's time to create a new Baddie ❻. When we've gone through the constant `ADD_NEW_BADDIE_RATE` number of frames, we create a new `Baddie` object and add it to the list of Baddies.

The `draw()` method ❼ iterates through the list of Baddies and calls the `draw()` method of each Baddie to draw itself at its appropriate location.

Finally, the `hasPlayerHitBaddie()` method ❽ checks to see if the Player's rectangle intersects any Baddie. The code iterates through the list of